MIDTERM STUDY GUIDE

Introduction to IoT and Edge Computing

Weeks 1-5 + Week 6 Practical

Condensed, comprehensive review for efficient midterm preparation

Physical world	Sense	Prepare data	Connect	Communicate	Process / Act

Important exam note

You will NOT be required to write complete Python programs from memory. For practical-related questions, you should understand what the program is doing, recognize the role of key functions, trace the data flow, compare TCP/UDP/MQTT behavior, and interpret short code fragments or outputs.

Prepared from the official Week 1-6 course materials.

How to Use This Guide

This guide is designed to replace a full re-reading of the long lecture decks during midterm revision. It keeps the course sequence, terminology, comparisons, and practical logic, but removes repeated examples and implementation detail that are not essential for the exam.

Study in three passes

Pass	What to do
1. Understand the journey	Learn the end-to-end IoT flow: physical quantity -> sensor -> IoT device -> structured/serialized data -> connectivity -> IP/transport -> application protocol -> processing/action.
2. Review each week	Focus on definitions, comparisons, trade-offs, and why a technology or protocol is chosen.
3. Test yourself	Use the final self-check section. If you cannot explain an answer in one or two sentences, return to the relevant section.

What “understand the Python practical” means

- You may be shown a short code fragment and asked what it does.
- You may be asked to identify which step converts a Python dictionary to JSON, text to bytes, or bytes back to text.
- You may be asked why TCP fails when the server is not listening, while a UDP sender may still report that it sent a datagram.
- You may be asked to explain publisher -> broker -> subscriber flow, or why the publisher does not need subscriber addresses.
- You may be asked to interpret Node-RED fields such as msg.topic and msg.payload.

Do not memorize syntax

Understand purpose and sequence. For example, know that json.dumps() serializes a Python dictionary into JSON text; you do not need to reproduce a complete program.

The Course as One End-to-End System

Physical quantity	Sensor	IoT device	Structured data	Serialized payload
Connectivity	**IP**	**TCP / UDP**	**HTTP / CoAP / MQTT**	**Application / action**

Each week answers a different engineering question:

Week	Main question	Core concepts
1	What is an IoT system?	Components, data flow, physical-digital loop, characteristics, challenges
2	How is physical information measured and interpreted over time?	Sensors, data acquisition, sampling, time-series data, data quality, aggregation
3	How does an IoT device prepare data?	Device anatomy, conversion, validation, structuring, serialization, JSON/binary
4	How does the device establish a communication link?	Connectivity requirements, technologies, topology, technology selection
5	Once connected, how is data addressed, transported, and exchanged?	IP, ports, TCP/UDP, request-response, HTTP, CoAP, MQTT
6 practical	How do these ideas behave in real programs?	Sockets, JSON transfer, MQTT broker, edge logic, Node-RED, virtual actuator

Key distinction

Week 3 prepares WHAT is sent. Week 4 establishes HOW the device connects. Week 5 defines HOW data is addressed, transported, and exchanged. Week 6 lets you observe those layers in practice.

Week 1 - Introduction to IoT

1. What is IoT?

Internet-of-Things (IoT) connects the physical world with digital systems. The important idea is not merely that a device is connected; the system uses data from physical things to monitor conditions, support decisions, issue alerts, or trigger actions.

Main IoT components

Component	Role
Physical object	The real-world thing or environment being observed or controlled.
Sensor	Measures a physical phenomenon and produces data.
IoT device	Provides sensing/processing/communication capability.
Network	Carries data or commands between system parts.
Processing system	Receives, analyzes, stores, or evaluates data.
Application	Makes information useful to users or other systems.
User or actuator	User views/controls the system; actuator changes the physical world.

A device is not the whole IoT system

> Example: Smart watch -> Smartphone -> Internet -> Cloud platform -> Health application -> User

A complete IoT system includes physical device, data, communication, processing, application, and a user or action.

Data flow vs control flow

Data generally moves from the physical world toward processing and applications. Commands/control can move in the opposite direction toward an actuator.

> Data: temperature -> sensor -> network -> application -> user
> Control: user/application -> network -> actuator -> physical action

Week 1 - How IoT Systems Behave

Physical-digital loop

Physical environment	Sense	Data	Process	Decision	Action

Many IoT systems operate continuously: the system observes the physical world, processes data, makes a decision, acts, and then observes the changed physical state again.

Traditional computing vs IoT

Traditional system	IoT system
Human often enters data	Sensors often generate data automatically
Mostly digital environment	Physical + digital interaction
Usually fewer computing devices	Potentially many devices
User initiates many actions	System may act automatically
Often centralized computing	Processing may be distributed

Core characteristics and design challenges

Concept	Why it matters
Scalability	More devices create more messages, storage, processing, management, and possible failures.
Continuous operation/data	Sensors may generate data continuously; system design must decide what to send and store.
Heterogeneity	Devices differ in sensors, hardware, operating systems, formats, connectivity, and capability.
Limited resources	Many devices have limited CPU, memory, storage, battery, and communication capacity.
Distributed processing	Different tasks may occur on device, nearby edge/gateway, or cloud.
Latency	Some applications require fast response; long cloud round trips may be unsuitable.
Bandwidth	Sending all raw data can create traffic, storage, and cost problems.
Network dependency	Internet failure can stop remote monitoring; local processing/storage may be needed.
Reliability	Sensor, communication, gateway, software, battery, or cloud failures can occur.
Security & privacy	IoT creates many attack points and may collect sensitive operational, location, health, or video data.

Week 2 - Sensors and Data Acquisition

Sensor vs actuator

Sensor	Actuator
Observes/measures the physical world.	Changes the physical world.
Examples: temperature sensor, camera, accelerometer, GPS.	Examples: motor, valve, fan, relay.
Physical condition -> sensor -> data	Command -> actuator -> physical action

Important sensor characteristics

Characteristic	Meaning	Typical question
Range	Minimum and maximum measurable values.	Can this sensor measure the expected operating condition?
Accuracy	Closeness to the true value.	How correct is the reading?
Precision	Consistency of repeated readings.	Are repeated readings similar?
Resolution	Smallest change the sensor can report.	How fine is the measurement detail?
Sensitivity	How strongly output changes when input changes.	How responsive is the output to physical change?
Sampling rate	How frequently measurements are collected.	How often must we measure this phenomenon?

A sensor can be precise but inaccurate: repeated readings may be very consistent but still far from the true value. Calibration is used to correct systematic error.

Sampling rate

Sampling rate should match how quickly the physical phenomenon changes. Slow phenomena such as room temperature can use slower sampling than high-frequency machine vibration.

Trade-off

Higher sampling rate captures faster changes but produces more data and can increase storage, network traffic, processing, and energy use.

Week 2 - Data Acquisition and Time-Series Data

Data acquisition

Physical measurement	Sensor signal	Sampling	Digital value	Timestamped record

Data acquisition is the process of obtaining usable digital data from a sensor. It can include reading the sensor, converting the signal, sampling, adding timestamp/device identity, and then storing or transmitting the measurement.

Typical IoT observation record

Field	Purpose	Example
device_id	Identifies source device	sensor_01
timestamp	Shows when measurement occurred	2026-09-08 10:32:15
sensor_type	Describes what was measured	temperature
value	Stores the measurement	25.4
unit	Gives the measurement unit	°C
status	Optional interpretation	normal / warning

Time-series data

Time-series data is data ordered by time. The timestamp allows the system to reconstruct trends, rates of change, missing measurements, late data, and sequence of events.

```
10:00  23.5°C
10:01  24.1°C
10:02  24.8°C
10:03  29.7°C
```

A single reading shows a moment. A sequence shows context, variability, trend, and possible anomalies.

Data volume from sampling

At one reading per second, one sensor produces 86,400 readings per day. At 1,000 sensors, this becomes 86,400,000 readings per day. Sampling design therefore affects system scale.

Week 2 - Data Quality, Aggregation, and Moving Average

Common data-quality problems

Problem	Meaning
Noise	Small random variation around the true physical value.
Outlier	An extreme or unusual value; may be an error or a real event.
Missing data	An expected measurement is absent.
Duplicate data	The same measurement/record appears more than once.
Delayed/out-of-order data	A measurement arrives later than newer measurements.
Wrong timestamp / drift	Recorded time is incorrect, or the sensor gradually becomes inaccurate.

Important

Do not automatically delete an unusual value. An outlier may indicate a true abnormal event, sensor failure, communication error, or environmental disturbance.

Aggregation

Aggregation summarizes many observations using statistics such as count, minimum, maximum, and average. It reduces data volume and makes trends easier to interpret.

> Readings: 23.5, 24.1, 24.8, 29.7, 25.2
> Minimum = 23.5°C | Maximum = 29.7°C | Average = 25.46°C

Moving average

A moving average calculates the mean of a fixed number of recent sequential points. As time moves forward, the oldest value leaves the window and the newest value enters.

> Window size 3: (t1+t2+t3)/3 -> (t2+t3+t4)/3 -> (t3+t4+t5)/3

Smaller window	Larger window
Responds faster to sudden changes.	Produces smoother output.
More sensitive to noise.	Filters short-term variation more strongly.
Less response lag.	More response lag.

Simple anomaly detection

A threshold rule such as temperature > 30°C -> WARNING is a simple form of anomaly detection. A pattern can also be abnormal, for example a steadily increasing temperature even before a fixed threshold is crossed.

Week 3 - IoT Devices

What is an IoT device?

An IoT smart object contains enough sensing, processing, and communication capability to participate in a digital system. Components vary with application requirements and device complexity.

Component	Function
Processor / MCU / SoC	Reads data, performs local processing, evaluates rules, prepares communication.
Memory	Temporary working area for running instructions, variables, and sensor buffers.
Storage	Keeps information longer until erased or overwritten.
Communication interface	Wi-Fi, Bluetooth, Ethernet, cellular, LoRa, Zigbee, etc.
Power subsystem	Battery, mains, solar, or another source.
Sensor interface	Receives measurement signals from sensors.
Actuator interface	Sends control signals toward physical action.

Analog vs digital sensor paths

Analog sensor path	Digital sensor path
Physical change -> voltage variation -> ADC in IoT device -> digital value	Physical change -> internal sensor conversion -> serial/digital transmission -> IoT device

ADC idea

An Analog-to-Digital Converter (ADC) converts an analog electrical signal into a digital number. For an n-bit ADC there are 2^n possible digital levels. Higher bit depth gives smaller voltage steps and finer digital resolution.

> Example: 12-bit ADC -> 2^12 = 4096 levels.
> Approximate step size = Vref / 4096.

Do not confuse two meanings of resolution

Sensor resolution is the smallest physical change the sensor can report. ADC resolution refers to the number/size of digital conversion levels.

Week 3 - Data Preparation Pipeline

Raw acquisition	Conversion	Validation	Cleaning / organization	Encoding	Transmission

Conversion

Converts raw values into usable engineering units or numerical types. Example: a raw string "25.4" may need float conversion before a numerical comparison can be performed.

Validation

Checks whether measurements fall within expected boundaries. Example: humidity should usually satisfy 0% <= humidity <= 100%. Invalid data can be tagged rather than automatically deleted.

Missing values

Missing means absence of knowledge; zero is a real measured value. Replacing missing data with 0 can distort averages or trigger incorrect decisions.

Strategy	Idea / trade-off
Keep missing as None/null	Preserves the fact that data is missing.
Discard record	Useful when complete frames are required, but valid data in the same frame may be lost.
Use previous valid value	Useful for slow-changing signals, but may become stale if the sensor remains offline.

Duplicate and delayed data

- Sequence numbers can help identify duplicate records.
- Measurement/event time = when the physical event was measured.
- Received/ingestion time = when the edge/broker/cloud received the data.
- If these differ greatly, records can arrive out of order; timestamps allow chronological reconstruction.

Week 3 - Structuring and Serialization

Dictionary / key-value structure

```
sensor_data = {
  "device_id": "sensor_01",
  "temperature": 25.4,
  "humidity": 61.2,
  "quality": "VALID"
}
```

A dictionary is an in-memory Python data structure. Keys describe the meaning of each value, making the record easier to interpret than a position-only list.

Nested data

Nested structures group related measurements under a parent key. This is useful when one device has several measurements plus device-level metadata such as battery.

Serialization

Serialization translates an in-memory object into a standardized text or byte representation that can be stored or transmitted. A Python dictionary cannot be sent directly as a network byte stream; it must be represented in a transferable format.

High-yield practical concept

Python dictionary != JSON. A dictionary is a Python object. JSON is a text-based serialized representation. json.dumps() converts a Python dictionary into JSON text; json.loads() converts JSON text back into a Python object.

JSON vs binary representations

Format	Strengths	Limitations
JSON	Human-readable, common across languages, easy to debug.	Field names/text add payload overhead; parsing costs computation.
Packed binary	Small payload, lower transmission time/energy.	Not human-readable; sender/receiver must agree on structure.
CBOR	Binary representation of JSON-like structures; more compact.	Less human-readable than JSON.
Protocol Buffers	Compact schema-based binary representation.	Requires a shared schema (.proto).

Week 4 - IoT Connectivity Fundamentals

Connectivity vs communication protocol

Connectivity	Communication protocol
How the device gets connected.	How data is exchanged once a connection exists.
Examples: Wi-Fi, BLE, Ethernet, LoRa, cellular.	Examples later: IP, TCP/UDP, HTTP/CoAP/MQTT.
Creates the communication link.	Defines addressing, delivery behavior, and application interaction.

Connectivity requirements

Requirement	Question to ask
Range	How far must the link reach?
Data rate	How much data must be transferred?
Power	Is the device battery-powered? How much radio-on time is acceptable?
Latency	How quickly must information arrive/be used?
Mobility	Must communication continue while the device moves?
Device density	How many devices share the infrastructure?
Infrastructure	Is there an AP, phone, gateway, cellular operator, cable, or other network available?
Environment	Are there walls, steel, machinery, underground conditions, or interference?
Reliability	Is occasional loss acceptable? Is connection availability critical?
Cost	What deployment, service, hardware, or maintenance cost is acceptable?

Core trade-off

A system rarely maximizes long range, high data rate, and very low power at the same time. Connectivity selection is a trade-off based on application requirements.

Range vs mobility

Range asks how far a link can reach. Mobility asks whether communication must remain available while the device changes location. A fixed sensor 3 km from a gateway has a range requirement; a scooter moving across a city has a mobility/coverage requirement.

Week 4 - Local and Short-Range Technologies

Technology	Core characteristics	Typical use
Ethernet	Wired, stable, high throughput, low RF interference; cabling required; no mobility.	Industrial machine, fixed camera, gateway.
Wi-Fi	Local-area, high throughput, common infrastructure; higher power than low-power IoT radios.	Camera, appliance, local high-data device.
BLE	Short range, very low power, small/moderate data; often phone/gateway nearby.	Wearable, nearby sensor.
IEEE 802.15.4	Low-rate, low-power wireless foundation, short/medium range, suitable for many sensor nodes.	Building/environmental sensor network.
RFID	Radio-based identification; can read tags without direct line of sight; multiple tags possible.	Inventory, asset identification.
NFC	Very short-range/near-field communication.	Close-proximity identification/payment-style interactions.

Topology

Topology	Meaning	Trade-off
Point-to-point	Two endpoints communicate directly.	Simple direct link.
Star	Many devices connect through one central node.	Simple management; central-node dependency; endpoints must reach central node.
Mesh	Nodes may relay through neighboring nodes.	Extended coverage/alternative paths; more complexity and relay energy.

Gateway idea

A gateway provides a connection or bridge between devices/local networks and external networks/services. The exact role depends on the system: it may forward data, connect different connectivity technologies, or provide nearby processing.

Week 4 - Wide-Area Connectivity

LPWAN

Low-Power Wide-Area Network (LPWAN) technologies are designed for long-range IoT communication with low power, usually at the cost of very low data rate.

Technology	What to remember
LoRa	Long-range, low-power radio modulation / physical-layer technology; very low data rate.
LoRaWAN	Defines how LoRa devices, gateways, and network services communicate in a complete network.
Cellular IoT	Uses mobile-operator infrastructure; wide coverage and mobility support; service/subscription dependency.
NB-IoT	Cellular LPWAN; low data rate and low power; suitable for relatively static, low-data devices; good indoor coverage.
LTE-M	Cellular IoT for connected/mobile things; higher data capability and better mobility than NB-IoT.
4G/5G	Wide-area, high data rate and strong mobility support, generally higher power.
Satellite	Very wide/global coverage where terrestrial networks are unavailable; higher cost/power constraints.

Technology selection framework

1. How far? 2. How much data? 3. How much power? 4. How quickly?
5. Moving or fixed? 6. How many devices? 7. What infrastructure exists?
8. What environment? 9. What reliability? 10. What cost?

Typical decisions

Scenario	Likely choice	Reasoning
Machine-vision camera	Ethernet (or Wi-Fi if cabling is difficult)	High data, fixed, power available, reliability important.
Wearable -> smartphone	BLE	Short range, small data, very low power, phone available.
Remote farm sensor	LoRa / suitable LPWAN	Long range, tiny periodic data, low power.
City-wide mobile tracker	LTE-M / cellular	Wide coverage, mobility, operator infrastructure.

Week 5 - Communication Stack

One protocol cannot do everything

Layer	Main question	Examples
Connectivity / Link	How does the device access the network?	Wi-Fi, Ethernet, IEEE 802.15.4
Network	Where should the data go?	IP
Transport	Which application/process should receive it? How should it be delivered?	TCP, UDP
Application	How should applications exchange information?	HTTP, CoAP, MQTT

Outgoing example: MQTT -> TCP -> IP -> Wi-Fi
Conceptual bottom-up view: Wi-Fi -> IP -> TCP -> MQTT

IP address

An IP address is a logical address used to identify a device/interface on an IP network. Connectivity provides access to the network; IP enables addressing and delivery across networks.

Concept	Key point
IPv4	32-bit address, commonly written as four decimal octets, e.g. 192.168.1.25.
Private IP	Used inside local networks and not directly routable on the public Internet.
Public IP	Used for globally routable Internet communication.
NAT	Translates between private and external addressing so local devices can communicate with Internet services.
IPv6	128-bit addressing with a much larger address space; written in hexadecimal groups.
Router/default gateway	Forwards traffic between different IP networks.

Week 5 - Constrained Networking, Ports, TCP, and UDP

6LoWPAN

6LoWPAN means IPv6 over Low-Power Wireless Personal Area Networks. It is an adaptation layer that enables IPv6 communication over constrained low-power wireless links.

- Header compression reduces communication overhead.
- Fragmentation and reassembly allow larger IPv6 packets to cross small-frame links.
- A border router/gateway connects the constrained network to a wider IPv6 network.

Ports

One host can run many services at the same IP address. A port number identifies the transport-layer application/process. IP address identifies the host/interface; port identifies the process/service.

> Endpoint idea: IP address + port
> Example: 127.0.0.1:1883 = MQTT broker on the same computer

TCP vs UDP

TCP	UDP
Connection-oriented; connection setup occurs before application data exchange.	Connectionless; no handshake is required before sending a datagram.
Reliable, ordered byte-stream transport.	Datagram transport with no built-in delivery or ordering guarantee.
Uses acknowledgments/retransmission and tracks communication state.	Lower transport complexity; sender may not know whether the receiving application obtained the datagram.
Typical when reliable ordered delivery is important.	Typical when the application accepts possible loss or needs simple datagram behavior.

TCP reliability concepts

- Sequence/order: data is reconstructed in the correct order.
- Acknowledgment: receiver confirms received data.
- Retransmission: sender can resend when expected acknowledgment does not arrive.
- Duplicate handling: duplicate transport data can be detected/ignored.

Week 5 - Request-Response: HTTP and CoAP

Why transport alone is not enough

TCP/UDP can move application data, but they do not define whether the data is a resource request, sensor measurement, command, or configuration change. Application protocols define interaction rules.

Request-response model

A client initiates a request for a resource/action; a server processes the request and sends a response.

```
Client -> request -> Server
Client <- response <- Server
```

HTTP

HTTP is an application-layer request-response protocol widely used for web/cloud/API systems. In the course examples, HTTP commonly operates over TCP/IP.

Method	Meaning in IoT example
GET	Read a sensor/status resource.
POST	Submit data or create an action/resource.
PUT	Update a resource or configuration.
DELETE	Remove a resource/configuration.
Status code	**Meaning**
200	OK
201	Created
400	Bad request
404	Not found
500	Server error

CoAP

CoAP (Constrained Application Protocol) is a lightweight, resource-oriented application protocol for constrained IoT environments. It uses a request-response model conceptually similar to HTTP and commonly uses UDP.

HTTP	CoAP
Request-response	Request-response
Commonly TCP in course examples	Commonly UDP
Text-oriented HTTP messages	Compact binary messages
Web/cloud/API systems	Constrained IoT

Week 5 - MQTT and Publish-Subscribe

Why publish-subscribe?

Continuous polling can create repeated requests and unnecessary traffic when many sensors and applications are involved. Publish-subscribe lets producers publish only when they have data, while subscribers receive matching messages through a broker.

Publisher	Topic	Broker	Matching subscribers

MQTT roles

Role	Meaning
Publisher	Produces a message and publishes it to a topic.
Broker	Receives publications and routes/distributes them by topic.
Subscriber	Subscribes to topic filters and receives matching messages.
Topic	Hierarchical classification/routing label.
Payload	Actual application-defined information carried by the message.

Publishers and subscribers do not need to know each other directly. The publisher mainly needs the broker address, topic, payload, and messaging settings.

MQTT over TCP/IP

In the course, MQTT typically operates over TCP/IP: first a TCP connection is established to the broker, then MQTT CONNECT/CONNACK occurs, then clients can PUBLISH and SUBSCRIBE.

Topic design

> Good: factory/line1/machine01/temperature
> Poor: Sensor1, Data3

Hierarchical topic names improve organization, scalability, and subscription management.

Week 5 - MQTT Wildcards, QoS, Retained Message, Last Will

Wildcards are used in subscription filters

Wildcard	Meaning	Example
+	Matches exactly one topic level.	factory/+/temperature
#	Matches zero or more remaining levels; used for a subtree.	factory/#

factory/+/temperature -> matches machine01 temperature, machine02 temperature, ...
factory/# -> matches everything under the factory branch

QoS - MQTT delivery assurance

QoS	Delivery behavior	Typical use / trade-off
0	At most once; no MQTT-level acknowledgment/redelivery.	Frequent non-critical telemetry; lowest overhead; possible loss.
1	At least once; PUBACK; retransmission can create duplicates.	Important telemetry/alerts; duplicates must be tolerated.
2	Exactly-once delivery semantics at MQTT protocol level; more exchanges.	When duplicate MQTT delivery should be avoided; highest overhead.

Retained message

The broker stores the latest retained message for a topic and delivers it to a new subscriber. It is the latest retained state, not complete message history.

Last Will

A Last Will is a predefined message configured when a client connects. The broker publishes it if that client disconnects unexpectedly. Example: topic factory/machine01/status, payload offline.

Do not confuse

Retained message = latest retained state for a topic. Last Will = predefined message published after an unexpected disconnect.

Week 5 - Selecting the Communication Approach

Question	If yes / likely choice
Need to read or modify a resource?	Request-response -> HTTP or CoAP.
Need continuous telemetry/events for multiple consumers?	Publish-subscribe -> MQTT.
Very constrained device/network?	Consider CoAP / UDP plus constrained networking support.
Web/cloud/API integration?	HTTP.
Brokered telemetry?	MQTT.
Need reliable ordered transport?	TCP.
Need lightweight datagram transport without built-in reliability?	UDP.
Need selectable MQTT publication assurance?	MQTT QoS 0/1/2.

Complete stack examples

Example A - Wi-Fi + MQTT telemetry

Sensor data	JSON	MQTT	TCP	IP	Wi-Fi

JSON solves data representation; Wi-Fi provides local connectivity; IP provides network addressing; TCP provides transport behavior; MQTT provides application-level publish-subscribe messaging.

Example B - Constrained room sensor

Application data	CoAP	UDP	IPv6	6LoWPAN	IEEE 802.15.4

6LoWPAN adapts IPv6 to the constrained low-power link. CoAP provides a lightweight resource-oriented application interaction.

Week 6 Practical - What You Need to Understand for the Exam

No raw-code writing requirement

You are expected to READ and INTERPRET practical code, not reproduce complete programs. Focus on data types, sequence, network behavior, and the role of important functions.

The practical progression

Python data	TCP sockets	UDP sockets	MQTT messaging	Edge logic	Virtual action

The purpose of the practical is to show how low-level transport communication becomes application-level IoT messaging and then a sense-process-act loop.

Three networking concepts

Concept	Meaning in the practical	Example
IP address	Identifies the host.	127.0.0.1 = localhost / same computer
Port	Identifies the application/process.	5000, 5001, 1883
Socket	Programming interface to TCP or UDP transport.	TCP socket or UDP socket

Recognize socket creation

```
socket.AF_INET   -> IPv4 addressing
socket.SOCK_STREAM -> TCP
socket.SOCK_DGRAM  -> UDP
```

So socket.socket(AF_INET, SOCK_STREAM) means an IPv4 TCP socket. socket.socket(AF_INET, SOCK_DGRAM) means an IPv4 UDP socket.

Week 6 Practical - TCP Code Flow

Conceptual server flow

Create socket	bind()	listen()	accept()	recv()	send response	close

Conceptual client flow

Create data	json.dumps()	Create socket	connect()	encode() + sendall()	recv() + decode()

What each step means

Function / step	Meaning
bind((HOST, PORT))	Associates the server socket with a local IP address and port.
listen()	Puts the TCP server into listening mode.
accept()	Waits for and accepts an incoming TCP connection.
connect((HOST, PORT))	Client establishes a TCP connection to a listening server.
json.dumps(data)	Serializes a Python dictionary into JSON text.
encode()	Converts text into bytes for network transmission.
sendall(bytes)	Sends bytes through the TCP connection.
recv(...)	Receives bytes from the connected peer.
decode()	Converts bytes back into text.
json.loads(text)	Parses JSON text back into a Python object/dictionary.

Key observation: server stopped

If the TCP server is not running/listening, the client connect() attempt fails. This directly demonstrates that TCP is connection-oriented: application data exchange requires an established connection.

Do not confuse

The server text response “Sensor data received” is an application-level response. It is not the same as internal TCP acknowledgment packets.

Week 6 Practical - UDP Code Flow and TCP/UDP Comparison

UDP receiver

Create UDP socket	bind()	recvfrom()	decode()	json.loads()	close

UDP sender

Create data	json.dumps()	Create UDP socket	encode() + sendto()	close

What matters

- No connect() or accept() is required for the basic UDP exchange.
- sendto() includes the destination IP + port.
- recvfrom() receives a datagram and also gives the sender address.
- UDP still uses IP addresses and ports. The major difference is transport behavior, not addressing.

Receiver stopped experiment

When the UDP receiver is stopped, the sender may still print that it sent the datagram. That does NOT prove a receiving application obtained it. UDP has no built-in delivery confirmation.

Same data, different transport

```
Python dictionary -> JSON string -> bytes -> TCP OR UDP
```

The application data and its meaning do not change. What changes is how the bytes are transported.

TCP	UDP
connect() / accept()	No connection setup
sendall() / recv()	sendto() / recvfrom()
Byte stream	Datagrams
Built-in reliability/order	No built-in delivery/order guarantee

Week 6 Practical - From Sockets to MQTT

Why MQTT after sockets?

Raw sockets show direct TCP/UDP communication using IP addresses, ports, and bytes. MQTT adds higher-level IoT messaging concepts such as publisher, subscriber, broker, topic, QoS, and retained messages. MQTT does not replace TCP; in the course it typically uses TCP/IP underneath.

MQTT practical architecture

Python sensor publisher	Mosquitto broker	Python subscriber / Node-RED

Publisher knows vs does not need to know

Publisher needs	Publisher does NOT need
Broker address / port	Subscriber IP address
Topic	How many subscribers exist
Payload	Which dashboard/database receives it
QoS/retain settings as required	Direct connection to every consumer

Subscriber concepts

Code idea	Meaning
client.subscribe(TOPIC)	Ask the broker to deliver messages matching the topic/filter.
on_connect(..., reason_code, ...)	Connection callback; reason_code indicates the broker connection result; subscription is commonly started here.
on_message(...)	Callback executed when a matching MQTT message arrives.
message.topic / message.payload.decode()	Read the routing topic and convert received payload bytes into text.
loop_forever()	Keep MQTT network processing running continuously.

Paho client setting: mqtt.CallbackAPIVersion.VERSION2 means the program uses Paho MQTT current Version 2 callback interface.

Publisher concepts

Code idea	Meaning
client.connect(BROKER, PORT, keepalive=60)	Connect MQTT client to the broker; keepalive helps maintain/detect the connection during quiet periods.
json.dumps(sensor_data)	Create a JSON text payload.
client.publish(TOPIC, payload, qos=...)	Publish the payload to a topic through the broker.
loop_start() + sensor loop	Run MQTT network processing in the background while the program continues generating readings; time.sleep(...) sets the practical sampling/publish interval.

Week 6 Practical - Edge Logic, Actuator, and Node-RED

Python edge processor

Subscribe to temperature topic	Decode + parse JSON	Read temperature	Apply threshold	Publish FAN ON/OFF

The edge processor acts as both an MQTT subscriber and publisher. It receives sensor data, applies local application logic, and publishes a command to another topic.

```
If temperature > 30°C -> FAN ON
Else -> FAN OFF
```

Virtual actuator

The actuator subscribes to the command topic and interprets the command payload. In the practical it prints ON/OFF; real hardware could replace the print statement.

Complete sense-process-act loop

Python sensor	Broker	Edge processor / Node-RED	Broker	Virtual actuator

Node-RED flow

MQTT In	JSON	Debug	Switch	Change	MQTT Out

Node	Purpose
MQTT In	Subscribe to sensor topic.
JSON	Parse JSON text so msg.payload becomes an object.
Debug	Display received data for observation/troubleshooting.
Switch	Evaluate msg.payload.temperature against threshold.
Change	Create FAN ON/FAN OFF command and set message topic.
MQTT Out	Publish command to the broker.

msg.topic vs msg.payload

Field	Meaning	Example
msg.topic	Label / routing category	factory/machine01/temperature
msg.payload	Actual message content	{"temperature":34.4,"unit":"C"}

Practical Code-Reading Cheat Sheet

Use this for questions like the revised mock-assignment Q13

You should be able to explain the difference between a Python dictionary and JSON, why serialization is required, and what key networking/MQTT functions do. You do not need to write a full program.

You see...	You should think...
sensor_data = { ... }	Python dictionary / structured in-memory application data.
json.dumps(sensor_data)	Serialize dictionary -> JSON string.
json.loads(message)	Parse JSON string -> Python object/dictionary.
message.encode()	Text -> bytes for transport.
data.decode()	Bytes -> text.
AF_INET	IPv4 addressing family.
SOCK_STREAM	TCP socket.
SOCK_DGRAM	UDP socket.
bind()	Assign local IP + port to a socket.
listen() / accept()	TCP server waits for and accepts a connection.
connect()	TCP client establishes a connection.
sendall() / recv()	Send/receive data on a connected TCP stream.
sendto() / recvfrom()	Send/receive UDP datagrams.
client.subscribe(topic)	MQTT subscriber asks broker for matching topic messages.
client.publish(topic,payload)	MQTT publisher sends message to broker/topic.
on_message(...)	Callback that runs when an MQTT message arrives.
msg.topic	Node-RED message routing/category field.
msg.payload	Node-RED actual message data field.

Commonly Confused Concepts

Do not confuse	Correct distinction
IoT device vs IoT system	A device is one component; the system includes data, communication, processing, application, and user/action.
Sensor vs actuator	Sensor observes; actuator changes the physical world.
Accuracy vs precision	Accuracy = closeness to true value; precision = repeatability.
Sampling rate vs data rate	Sampling rate = how often measurements are taken; data rate = how fast communication transfers bits.
Missing value vs zero	Missing = no knowledge; zero can be a real measurement.
Python dictionary vs JSON	Dictionary is an in-memory Python structure; JSON is serialized text representation.
Connectivity vs protocol	Connectivity establishes a link; protocols define addressing, transport, and application interaction.
Range vs mobility	Range = link distance; mobility = continued communication while moving.
LoRa vs LoRaWAN	LoRa = radio modulation/physical layer; LoRaWAN = network protocol/system using LoRa.
IP vs port	IP identifies host/interface; port identifies application/process.
TCP vs application response	TCP reliability is transport-level; an application response is generated by the program/protocol.
TCP vs UDP	TCP is connection-oriented reliable ordered byte stream; UDP is connectionless datagram transport without built-in delivery/order guarantee.
HTTP/CoAP vs MQTT	HTTP/CoAP use request-response; MQTT uses brokered publish-subscribe.
MQTT topic vs payload	Topic is classification/routing label; payload is actual content.
QoS 0 vs UDP	MQTT QoS 0 is still MQTT and normally runs over TCP in this course; it does not mean UDP.
Retained vs Last Will	Retained = latest stored topic state for new subscriber; Last Will = predefined unexpected-disconnect message.
Node-RED vs MQTT	Node-RED applies/visualizes application logic; MQTT provides brokered messaging.

How to Answer Scenario Questions

1. Sensor selection

> What physical quantity? -> required range/accuracy/sampling -> environment/power -> choose sensor -> justify

2. Connectivity selection

> Range + data + power + latency + mobility + infrastructure + environment + reliability/cost -> choose technology -> explain trade-off

3. Communication protocol selection

> Interaction model -> constrained or not -> transport behavior -> application protocol

- Read/change a resource -> request-response -> HTTP or CoAP.
- Continuous telemetry to multiple consumers -> publish-subscribe -> MQTT.
- Reliable ordered transport -> TCP; simple datagrams -> UDP.

4. Code-reading question

> Identify data type -> identify operation -> state what changes -> explain communication effect / expected output

Example reasoning: sensor_data is a Python dictionary. json.dumps() serializes it into JSON text. encode() converts text to bytes so the transport can send it. The receiver decode() converts bytes back to text and json.loads() reconstructs a Python object.

5. End-to-end trace

> Physical value -> Sensor -> IoT device -> Structured data -> JSON -> Connectivity -> IP -> TCP/UDP -> HTTP/CoAP/MQTT -> Application -> Decision/action

Self-Check Questions

Try to answer without looking back. These are review questions, not copies of the real midterm.

1. Why is a smart device alone not necessarily a complete IoT system?
2. Explain the difference between a sensor and an actuator.
3. A sensor gives nearly the same wrong reading every time. Is it accurate, precise, both, or neither?
4. Why does a faster sampling rate usually increase data volume?
5. What is the difference between noise and an outlier?
6. Why should a missing temperature value not automatically be replaced by 0?
7. Explain Python dictionary vs JSON in one or two sentences.
8. Why is serialization needed before network transmission?
9. What is the difference between connectivity and a communication protocol?
10. Which connectivity technology is a good fit for a battery wearable communicating with a nearby smartphone, and why?
11. Which technology is a likely fit for a fixed high-data machine-vision camera?
12. Explain LoRa vs LoRaWAN.
13. What does an IP address identify? What does a port identify?
14. Why can a TCP client fail immediately when the server is stopped?
15. Why can a UDP sender report “sent” even when no receiver application is running?
16. What is the difference between request-response and publish-subscribe?
17. In MQTT, what are the roles of publisher, broker, subscriber, topic, and payload?
18. What does MQTT wildcard + match? What does # match?
19. Compare MQTT QoS 0 and QoS 1.
20. Explain retained message vs Last Will.
21. What does json.dumps() do? What do encode() and decode() do?
22. What do AF_INET, SOCK_STREAM, and SOCK_DGRAM mean?
23. In Node-RED, how do msg.topic and msg.payload differ?
24. Trace the Week 6 control loop from Python sensor to FAN ON/OFF.
25. Why does MQTT make it easier to add more subscribers than a direct point-to-point design?

Self-Check Answers

1. A complete IoT system also needs data exchange, communication, processing, an application, and a user or action.

2. A sensor observes/measures the physical world; an actuator performs a physical action based on a command.

3. Precise but not accurate.

4. More measurements are created per unit time, so more records must be processed/stored/transmitted.

5. Noise is small random variation; an outlier is an unusual extreme value that may be error or a real event.

6. Zero is a real physical value; missing means no measurement/knowledge. Replacing missing with zero can corrupt analysis or decisions.

7. A dictionary is an in-memory Python key-value object. JSON is a standardized text representation used to store/transmit structured data.

8. Network transmission requires a text/byte stream rather than a language-specific in-memory object.

9. Connectivity establishes the link; protocols define how data is addressed, transported, and exchanged.

10. BLE: short range, small data, very low power, and the smartphone can act as the nearby central/gateway.

11. Ethernet, or Wi-Fi if cabling is difficult: high data rate and power are available; the device is fixed.

12. LoRa is the long-range low-power radio modulation; LoRaWAN defines network communication among devices, gateways, and network services.

13. IP identifies a host/interface on the network; a port identifies the application/process on that host.

14. TCP requires a connection to a listening endpoint before application data can be exchanged.

15. UDP has no built-in connection or delivery confirmation, so sending does not prove a receiver application obtained the datagram.

16. Request-response: client asks and server replies. Publish-subscribe: publisher sends to a topic/broker and matching subscribers receive.

17. Publisher produces; broker routes; subscriber receives; topic classifies/routes; payload contains the actual data.

18. + matches exactly one topic level. # matches zero or more remaining levels of a topic subtree.

19. QoS 0: no MQTT-level acknowledgment/redelivery. QoS 1: acknowledged, at least once, possible duplicates.

20. Retained stores the latest retained topic state for new subscribers; Last Will is a predefined message published after unexpected disconnect.

21. json.dumps(): dictionary -> JSON string. encode(): text -> bytes. decode(): bytes -> text.

22. AF_INET = IPv4; SOCK_STREAM = TCP; SOCK_DGRAM = UDP.

23. msg.topic is routing/category information; msg.payload is the actual message content.

24. Python sensor publishes temperature -> broker -> edge processor/Node-RED parses and checks threshold -> publishes FAN ON/OFF command -> broker -> virtual actuator receives command.

25. The publisher communicates with the broker/topic rather than maintaining direct connections to every consumer; the broker distributes to any matching subscribers.

Final 15-Minute Revision Checklist

You should be able to...	Check
Explain IoT and list the main system components.	☐
Trace data flow and control/action flow.	☐
Explain scalability, heterogeneity, limited resources, latency, bandwidth, reliability, security/privacy.	☐
Differentiate sensor vs actuator; accuracy vs precision; range/resolution/sensitivity/sampling rate.	☐
Explain time-series data, timestamps, data quality, aggregation, moving average, threshold anomaly.	☐
Explain IoT device anatomy and analog/digital sensor paths; basic ADC idea.	☐
Explain conversion, validation, missing/duplicate/delayed data, structuring, serialization.	☐
Differentiate Python dictionary, JSON, and binary representations.	☐
Select connectivity using range/data/power/latency/mobility/infrastructure/etc.	☐
Compare Ethernet, Wi-Fi, BLE, IEEE 802.15.4, RFID/NFC, LoRa/LoRaWAN, NB-IoT, LTE-M/cellular.	☐
Explain point-to-point, star, and mesh topology.	☐
Explain the protocol stack: connectivity -> IP -> TCP/UDP -> HTTP/CoAP/MQTT.	☐
Explain IPv4/IPv6/private/public/NAT/6LoWPAN at course level.	☐
Differentiate TCP and UDP and interpret the Week 6 experiments.	☐
Explain request-response, HTTP, and CoAP.	☐
Explain MQTT publisher/broker/subscriber, topics, wildcards, QoS, retained, Last Will.	☐
Read short Python snippets and explain json.dumps/loads, encode/decode, connect/send/recv, subscribe/publish.	☐
Explain the Week 6 sense-process-act loop and Node-RED msg.topic/msg.payload.	☐

Best final strategy

Do not memorize isolated definitions only. Practice explaining why each layer exists and how one sensor reading moves from the physical world to an application or actuator.

End of Midterm Study Guide